

Livro de Programação Competitiva — OBI

Módulo 1 — Estruturas Fundamentais

Objetivo do módulo: possuir todas as ferramentas básicas que sustentam tudo que vem depois (busca, técnicas, estruturas avançadas). Sempre que esquecer algo aqui, volte e releia em 30 minutos.

1.A — Pilhas (Stack)

1. Introdução

Objetivo: Depois desta seção você será capaz de reconhecer quando um problema pede uma pilha, implementá-la do zero e usar a STL corretamente, incluindo a técnica de *monotonic stack*.

Pré-requisitos: vetores, loops.

Quando estudar: antes de DFS iterativo (DFS usa pilha por baixo dos panos) e antes de qualquer problema com "próximo maior/menor elemento".

Nível: ★ Fácil (uso básico) → ★ ★ Médio (monotonic stack)

2. Intuição

Pense numa pilha de pratos. Você só consegue tirar o prato do **topo**. Para pegar um prato do meio, precisa tirar todos os de cima primeiro. Isso é LIFO: *Last In, First Out* — o último que entrou é o primeiro que sai.

3. Problema que motivou

Você tem uma sequência de expressões $((()))()$ e precisa saber se os parênteses estão balanceados. Ou: você tem uma sequência de números e, para cada um, quer saber qual é o próximo à direita que é maior que ele. Em ambos os casos, você precisa "lembrar" do que veio antes, na ordem inversa — pilha resolve isso naturalmente.

4. Construindo a solução

Solução 1 (força bruta) — validar parênteses: para cada $)$, procurar para trás qual $($ "fecha" com ele $\rightarrow O(N^2)$ no pior caso.

Solução final: empilhar $($ e, ao ver $)$, desempilhar. Se a pilha estiver vazia quando precisar desempilhar, está desbalanceado. No fim, a pilha deve estar vazia. $\rightarrow O(N)$.

O que mudou: em vez de procurar, a pilha já "guarda" o candidato mais recente pendente, que é sempre o certo a fechar primeiro.

5. Estrutura do algoritmo

```
push(x)  → coloca x no topo
pop()    → remove o elemento do topo (não retorna valor na STL)
top()    → olha o elemento do topo sem remover
empty()  → true se vazia
```

6. Implementação manual (com vetor)

```
cpp

const int MAXN = 100005;
int stk[MAXN];
int topo = -1; // pilha vazia

void push(int x) { stk[++topo] = x; }
void pop() { topo--; }
int top() { return stk[topo]; }
bool empty() { return topo == -1; }
```

7. Implementação STL

```
cpp

#include <stack>
using namespace std;

stack<int> st;
st.push(10);
st.push(20);
cout << st.top(); // 20
st.pop();
cout << st.top(); // 10
cout << st.empty(); // 0 (false)
```

8. Complexidade

Operação	Complexidade
push	O(1)
pop	O(1)
top	O(1)

Operação	Complexidade
empty	O(1)

9. Visualizações

```

push(5) push(7) push(9)

topo → [9]
      [7]
      [5]

pop() remove o 9:

topo → [7]
      [5]

```

10. Erros clássicos

- Chamar `top()` ou `pop()` numa pilha vazia (comportamento indefinido — sempre confira `empty()` antes)
- Achar que `pop()` retorna o valor removido — na STL, **não retorna**. Se precisa do valor, salve com `top()` antes de dar `pop()`.
- Usar pilha quando na verdade a ordem que importa é FIFO (aí é fila, não pilha)

11. Problemas clássicos (ordem crescente)

1. Validar parênteses balanceados (fácil)
2. Simplificar caminho de diretório tipo `/a/./b/./../c/` (médio)
3. Maior retângulo em histograma — usa monotonic stack (difícil)

12. Padrões — quando pensar em pilha

Se o enunciado diz...	Pense em...
Parênteses/colchetes balanceados	Pilha
"Próximo maior/menor elemento à direita/esquerda"	Monotonic Stack
Desfazer última ação (undo)	Pilha
Avaliar expressão matemática (notação pós-fixa)	Pilha
DFS sem recursão	Pilha (stack explícita)

13. Comparações

Pilha vs Fila: pilha é LIFO (último entra, primeiro sai); fila é FIFO (primeiro entra, primeiro sai). Pilha "lembra do mais recente"; fila "lembra do mais antigo pendente".

14. Casos onde NÃO usar

- Quando você precisa processar elementos na ordem de chegada (isso é fila)
- Quando precisa de acesso aleatório a qualquer posição (use vetor)

15. Problemas da OBI

- Problemas de simulação de "empilhar/desempilhar" caixas ou processos (buscar por "pilha" no NepsAcademy, filtro Programação Júnior/N1)

16. Outros problemas

- CSES: "Stack Sequences" (categoria Introductory)
- LeetCode: "Valid Parentheses" (fácil), "Daily Temperatures" (monotonic stack, médio)

17. Cheatsheet

```
cpp

stack<int> st;
st.push(x);
st.pop();      // não retorna valor
st.top();      // olha o topo
st.empty();    // true se vazia
st.size();     // quantidade de elementos
```

18. STL — Monotonic Stack (técnica, não estrutura nova)

Ideia: manter a pilha sempre crescente (ou decrescente). Antes de empilhar um novo elemento, desempilhe tudo que "quebraria" a monotonicidade.

```
cpp
```

```
// Exemplo: para cada elemento, achar o próximo maior à direita
vector<int> a = {2, 1, 2, 4, 3};
vector<int> resposta(a.size(), -1);
stack<int> st; // guarda ÍNDICES

for (int i = 0; i < (int)a.size(); i++) {
    while (!st.empty() && a[st.top()] < a[i]) {
        resposta[st.top()] = a[i];
        st.pop();
    }
    st.push(i);
}
// resposta = [4, 2, 4, -1, -1]
```

Complexidade $O(N)$ total, porque cada índice entra e sai da pilha no máximo uma vez.

19. Template

```
cpp

void resolverComPilha(vector<int>& a) {
    stack<int> st;
    for (int x : a) {
        while (!st.empty() && /* condição de desempilhar */ false) {
            st.pop();
        }
        st.push(x);
    }
}
```

20. Desafios

1. Valide se uma string de parênteses `(((){}[]))` está balanceada.
2. Dado um vetor, ache para cada elemento o próximo elemento à esquerda que é menor.
3. Simplifique um caminho de arquivo estilo Unix.
4. Ache o maior retângulo possível num histograma.
5. Avalie uma expressão em notação pós-fixa (RPN).

21. Mini-projeto

Simule uma pilha de "desfazer" (undo) de um editor de texto simples: cada operação (inserir caractere) é empilhada; um comando "undo" desempilha a última.

22. Revisão

- ☒ Por que pilha é LIFO e não FIFO?
- ☒ Por que `pop()` da STL não retorna valor?
- ☒ Quando uma monotonic stack processa cada elemento só uma vez, qual é a complexidade total?

23. Flashcards

Q: Quando usar monotonic stack? **A:** Quando o problema pede "próximo maior/menor elemento" em $O(N)$ total.

Q: O que acontece se você chamar `pop()` numa pilha vazia? **A:** Comportamento indefinido — sempre verifique `empty()` antes.

24. Resumo final

Pilha = LIFO. Use quando a ordem "mais recente primeiro" importa: parênteses, undo, DFS iterativo, monotonic stack para "próximo maior/menor". Complexidade $O(1)$ por operação. Erro clássico: esquecer de checar `empty()`.

25. Checklist de maestria

Conhecimento

- ☐ Sei explicar por que pilha é LIFO sem consultar material
- ☐ Sei identificar problemas que pedem pilha
- ☐ Conheço as limitações (não serve para FIFO, não tem acesso aleatório)

Implementação

- ☐ Consigo implementar pilha do zero com vetor
- ☐ Consigo usar `stack` da STL sem erro
- ☐ Sei implementar monotonic stack

Reconhecimento

- ☐ Reconheço "próximo maior/menor elemento" como monotonic stack
- ☐ Reconheço parênteses balanceados como problema de pilha

Competição

- ☐ Resolvi pelo menos 3 problemas fáceis de pilha
 - ☐ Resolvi pelo menos 1 problema de monotonic stack
-

1.B — Filas (Queue, Deque, Priority Queue, Heap)

1. Introdução

Objetivo: dominar fila simples (base do BFS), deque (base do 0-1 BFS e sliding window) e priority queue/heap (base do Dijkstra e de problemas gulosos).

Pré-requisitos: vetores, pilha (para contraste).

Quando estudar: logo antes de BFS — é a estrutura que sustenta todo o Módulo 2.

Nível: ★ Fácil (queue) → ★ ★ Médio (deque, priority_queue com comparador customizado)

2. Intuição

Queue: fila de banco — quem chegou primeiro é atendido primeiro (FIFO). **Deque:** uma fila que você pode inserir/remover **dos dois lados**. Pense numa fila de ônibus onde, em casos excepcionais, alguém pode ser colocado na frente. **Priority Queue / Heap:** uma fila de pronto-socorro — não importa quem chegou primeiro, quem está mais grave (maior prioridade) é atendido primeiro.

3. Problema que motivou

Você tem um grafo e quer visitar todos os nós "em ondas" a partir de uma origem, garantindo que a primeira vez que alcança um nó é pela distância mínima (queue → BFS). Depois, você tem pesos diferentes nas arestas e precisa sempre processar o nó de menor custo acumulado primeiro (priority_queue → Dijkstra).

4. Construindo a solução

Ver Módulo 2 (BFS e Dijkstra) para a construção completa — aqui o foco é a estrutura de dados em si.

5. Estrutura do algoritmo

```
Queue:          push_back → [ ] → pop_front
Deque:   push_front → [ ] ← push_back
          (pop dos dois lados também)
Priority Queue: sempre entrega o de MAIOR prioridade (por padrão, o maior valor)
```

6. Implementação manual (queue com vetor + dois ponteiros)

```
cpp
```

```

const int MAXN = 100005;
int arr[MAXN];
int inicio = 0, fim = 0;

void push(int x) { arr[fim++] = x; }
void pop() { inicio++; }
int front() { return arr[inicio]; }
bool empty() { return inicio == fim; }

```

7. Implementação STL

cpp

```

#include <queue>
#include <deque>
using namespace std;

// Queue
queue<int> q;
q.push(5); q.push(3);
q.front(); // 5
q.pop();

// Deque
deque<int> dq;
dq.push_back(1);
dq.push_front(2);
dq.front(); // 2
dq.back(); // 1
dq.pop_front();
dq.pop_back();

// Priority Queue (max-heap por padrão)
priority_queue<int> pq;
pq.push(3); pq.push(9); pq.push(1);
pq.top(); // 9 (o maior)
pq.pop();

// Priority Queue como MIN-heap (essencial para Dijkstra)
priority_queue<int, vector<int>, greater<int>> minpq;
minpq.push(3); minpq.push(9); minpq.push(1);
minpq.top(); // 1 (o menor)

```


8. Complexidade

Operação	Queue	Deque	Priority Queue
push/pop	$O(1)$	$O(1)$	$O(\log N)$
front/top	$O(1)$	$O(1)$	$O(1)$

9. Visualizações

Queue: [5][7][9] push(3) → [5][7][9][3]
 ↑ front remove daqui

Heap (max, como árvore):



10. Erros clássicos

- Confundir `front()` (queue, olha o início) com `top()` (priority_queue/stack, olha o "maior"/topo)
- Esquecer que `priority_queue` por padrão é **max-heap** — para Dijkstra você precisa do min-heap: `priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>>`
- Chamar `pop()`/`front()` em estrutura vazia
- Em Dijkstra, esquecer que a `priority_queue` pode ter entradas "desatualizadas" (o mesmo nó inserido mais de uma vez com distâncias diferentes) — sempre verificar se a distância ainda é válida ao processar

11. Problemas clássicos

1. Simular fila de atendimento (fácil)
2. Menor caminho em grid sem peso (BFS) — médio
3. K-ésimo maior elemento com stream de números (priority queue) — médio
4. Menor caminho com pesos (Dijkstra) — difícil

12. Padrões — quando pensar em cada estrutura

Se o enunciado diz...	Pense em...
Processar na ordem de chegada	Queue
Precisa inserir/remover nos dois extremos	Deque
"Sempre pegar o maior/menor disponível até agora"	Priority Queue
Pesos 0 e 1 nas arestas	Deque (0-1 BFS)
Pesos positivos quaisquer	Priority Queue (Dijkstra)
Top-K elementos de um fluxo de dados	Priority Queue

13. Comparações

Queue vs Deque: queue só mexe nas pontas fixas (insere no fim, remove do início); deque mexe nos dois lados livremente. Toda queue pode ser feita com deque, mas nem todo problema de deque pode ser feito com queue simples.

Priority Queue vs Queue: queue respeita ordem de chegada; priority_queue ignora ordem de chegada e sempre prioriza por valor.

14. Casos onde NÃO usar

- Priority Queue quando você precisa de ordem FIFO estrita (ela vai "furar a fila" por prioridade)
- Queue simples quando as arestas têm pesos diferentes (aí BFS simples dá resposta errada — precisa de Dijkstra)

15. Problemas da OBI

- Problemas de simulação com fila de atendimento/processos
- Problemas de "menor caminho" em grid (ver Módulo 2)

16. Outros problemas

- CSES: "Traffic Lights" (deque/multiset), "Sliding Window Median"
- LeetCode: "Sliding Window Maximum" (deque monotônico), "Kth Largest Element in a Stream" (priority_queue)

17. Cheatsheet

cpp

```
queue<int> q;
q.push(x); q.pop(); q.front(); q.back(); q.empty(); q.size();

deque<int> dq;
dq.push_front(x); dq.push_back(x);
dq.pop_front(); dq.pop_back();
dq.front(); dq.back();

priority_queue<int> pq; // max-heap
priority_queue<int, vector<int>, greater<int>> pq2; // min-heap
pq.push(x); pq.pop(); pq.top();
```

18. STL — Priority Queue com pares e structs

cpp

```
// min-heap de pares (usado em Dijkstra: {distância, nó})
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;

// com struct e comparator customizado
struct Tarefa {
    int prioridade, id;
};
struct Comparador {
    bool operator()(const Tarefa& a, const Tarefa& b) {
        return a.prioridade > b.prioridade; // menor prioridade primeiro
    }
};
priority_queue<Tarefa, vector<Tarefa>, Comparador> pq3;
```

19. Template

cpp

```
// BFS genérico usando queue – ver Módulo 2 para o template completo
queue<int> q;
q.push(origem);
while (!q.empty()) {
    int u = q.front(); q.pop();
    // processar u
}
```

20. Desafios

1. Implemente uma fila usando dois vetores (sem deslocar índices).
2. Simule uma fila de atendimento com prioridade (VIP furando fila).
3. Ache a mediana de uma sequência de números que chega em tempo real (dois heaps).
4. Implemente sliding window maximum com deque.
5. Implemente 0-1 BFS usando deque.

21. Mini-projeto

Sistema de fila de hospital: pacientes chegam com uma prioridade (1 a 5); simule o atendimento sempre pegando o de maior prioridade, e em empate, quem chegou primeiro.

22. Revisão

- ☒ Por que `priority_queue` por padrão é max-heap?
- ☒ Como transformar em min-heap?
- ☒ Por que BFS usa queue e não priority_queue?

23. Flashcards

Q: Qual estrutura usar para 0-1 BFS? A: Deque — pesos 0 vão para a frente, pesos 1 vão para trás.

Q: Como declarar uma priority_queue de menor primeiro? A: `priority_queue<int, vector<int>, greater<int>> pq;`

24. Resumo final

Queue = FIFO, base do BFS. Deque = fila com acesso nos dois lados, base do 0-1 BFS e sliding window. Priority Queue = sempre entrega o de maior prioridade (ou menor, com `greater<>`), base do Dijkstra e problemas gulosos. Todas $O(1)$ por operação, exceto priority_queue que é $O(\log N)$.

25. Checklist de maestria

Conhecimento

- ☐ Sei explicar quando usar queue vs deque vs priority_queue
- ☐ Sei por que BFS precisa de queue (não pilha)
- ☐ Sei por que Dijkstra precisa de priority_queue (não queue simples)

Implementação

- ☐ Consigo declarar min-heap e max-heap sem erro
- ☐ Consigo implementar priority_queue com comparador customizado

- ☐ Reconheço "pesos 0 e 1" como sinal de deque/0-1 BFS
- ☐ Reconheço "sempre o maior/menor disponível" como priority_queue

Competição

- ☐ Resolvi pelo menos 3 problemas com queue/BFS
 - ☐ Resolvi pelo menos 1 problema com priority_queue
-

1.C — Vetores e STL de Algoritmos

1. Introdução

Objetivo: dominar as ferramentas de `<algorithm>` que aparecem em praticamente toda prova: `sort`, `lower_bound`, `upper_bound`, `binary_search`, além de `pair`, `tuple`, `struct` e comparators customizados.

Pré-requisitos: vetores, noção de ordenação.

Quando estudar: antes de Binary Search (Módulo 3) e antes de qualquer problema com "ordene e depois processe".

Nível: ★ Fácil (sort básico) → ★★ Médio (comparador customizado, structs)

2. Intuição

Ordenar é, sozinho, uma das operações mais poderosas em programação competitiva: depois de ordenado, muitos problemas viram "percorra e some" ou "faça busca binária". `lower_bound`/`upper_bound` são a forma rápida ($O(\log N)$) de perguntar "onde esse valor encaixaria num vetor ordenado?".

3. Problema que motivou

Você tem uma lista de N preços e precisa, para Q consultas, responder "quantos produtos custam menos que X ?" rapidamente. Fazer uma busca linear para cada consulta é $O(N)$ por consulta. Se o vetor estiver ordenado, `lower_bound` responde em $O(\log N)$.

4. Construindo a solução

Solução 1 (força bruta): para cada consulta, percorra o vetor inteiro contando quantos são menores → $O(N)$ por consulta, $O(N \cdot Q)$ total.

Solução final: ordene o vetor uma vez ($O(N \log N)$), depois use `lower_bound` para cada consulta ($O(\log N)$) → $O(N \log N + Q \log N)$ total.

5. Estrutura do algoritmo

```
sort(begin, end)           → ordena o intervalo
lower_bound(begin, end, x)  → primeiro elemento >= x
upper_bound(begin, end, x)  → primeiro elemento > x
binary_search(begin, end, x) → true/false se x existe
```

6. Implementação manual (busca binária, para entender o que o STL faz por trás)

```
cpp

int lowerBoundManual(vector<int>& a, int x) {
    int lo = 0, hi = a.size(); // hi é exclusivo
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] < x) lo = mid + 1;
        else hi = mid;
    }
    return lo; // índice do primeiro elemento >= x
}
```

7. Implementação STL

```
cpp

#include <algorithm>
using namespace std;

vector<int> a = {5, 1, 4, 2, 3};
sort(a.begin(), a.end());           // {1,2,3,4,5}
sort(a.begin(), a.end(), greater<int>()); // decrescente

int x = 3;
auto it = lower_bound(a.begin(), a.end(), x); // aponta para o 3
int pos = it - a.begin();                     // índice = 2

auto it2 = upper_bound(a.begin(), a.end(), x); // aponta para o 4
bool existe = binary_search(a.begin(), a.end(), x); // true
```

pair, tuple, struct e comparator

```
cpp
```

```

// pair – útil para (peso, nó), (x, y), etc.
pair<int,int> p = {3, 5};
cout << p.first << " " << p.second;

vector<pair<int,int>> v = {{3,1}, {1,2}, {2,0}};
sort(v.begin(), v.end()); // ordena por .first, depois por .second

// tuple – quando precisa de 3+ campos
tuple<int,int,int> t = {1, 2, 3};
auto [a1, b1, c1] = t; // structured bindings (C++17)

// struct com comparador customizado
struct Pessoa {
    string nome;
    int idade;
};

bool comparaPorIdade(const Pessoa& a, const Pessoa& b) {
    return a.idade < b.idade; // crescente
}

vector<Pessoa> pessoas = {"Ana", 30}, {"Beto", 20};
sort(pessoas.begin(), pessoas.end(), comparaPorIdade);

// comparador como lambda (mais comum em prova)
sort(pessoas.begin(), pessoas.end(), [](const Pessoa& a, const Pessoa& b) {
    return a.idade < b.idade;
});

```

8. Complexidade

Operação	Complexidade
sort	$O(N \log N)$
lower_bound / upper_bound / binary_search	$O(\log N)$ — requer vetor já ordenado

9. Visualizações

```

Vetor ordenado: [1, 2, 3, 3, 3, 5, 8]
                  ↑lower_bound(3)  ↑upper_bound(3)
                  (primeiro 3)      (primeiro depois dos 3s, o 5)

```

10. Erros clássicos

- Usar `lower_bound`/`upper_bound`/`binary_search` num vetor **não ordenado** — resultado incorreto e silencioso (não dá erro de compilação nem crash)
- Confundir `lower_bound` (`>=`) com `upper_bound` (`>`)
- Esquecer de subtrair `a.begin()` para converter iterador em índice
- Comparator que não é uma "ordem estrita fraca" (ex: usar `<=` em vez de `<`) pode causar comportamento indefinido no `sort`

11. Problemas clássicos

1. Ordenar e imprimir (fácil)
2. Contar quantos elementos são menores que X, para várias consultas (médio)
3. Ordenar structs por múltiplos critérios com comparator customizado (médio)

12. Padrões — quando pensar nisso

Se o enunciado diz...	Pense em...
"Quantos elementos são menores/maiores que X"	Ordenar + <code>lower_bound</code> / <code>upper_bound</code>
"Ordene por critério A, empate por critério B"	Comparator customizado
"Existe algum elemento igual a X"	<code>binary_search</code> (com vetor ordenado)
Pares (x, y) que precisam ser processados juntos	<code>pair</code> /tuple

13. Comparações

`lower_bound` vs `upper_bound`: `lower_bound` acha o primeiro `>= x`; `upper_bound` acha o primeiro `> x`. A diferença entre os índices dá a **quantidade de ocorrências de x** no vetor.

14. Casos onde NÃO usar

- `lower_bound`/`upper_bound` em vetor não ordenado (sempre ordene antes)
- `sort` quando a ordem original importa e não pode ser alterada (nesse caso, ordene índices, não os valores)

15. Problemas da OBI

- Problemas que pedem "ordene e depois processe" — muito comuns nas fases iniciais (Programação Júnior/N1)

16. Outros problemas

- CSES: "Distinct Numbers", "Sorting and Searching" (categoria inteira)
- LeetCode: "Search Insert Position" (lower_bound direto)

17. Cheatsheet

cpp

```
sort(v.begin(), v.end());
sort(v.begin(), v.end(), greater<int>());
sort(v.begin(), v.end(), [](auto&a, auto&b){ return a.x < b.x; });

lower_bound(v.begin(), v.end(), x) - v.begin(); // índice do primeiro >= x
upper_bound(v.begin(), v.end(), x) - v.begin(); // índice do primeiro > x
binary_search(v.begin(), v.end(), x);           // true/false
```

18. STL — funções úteis extras

cpp

```
reverse(v.begin(), v.end());
int mn = *min_element(v.begin(), v.end());
int mx = *max_element(v.begin(), v.end());
long long soma = accumulate(v.begin(), v.end(), 0LL);
v.erase(unique(v.begin(), v.end()), v.end()); // remove duplicatas (vetor deve
```

19. Template

cpp

```
vector<int> v = /* ler entrada */;
sort(v.begin(), v.end());
for (int query : queries) {
    int qtd_menores = lower_bound(v.begin(), v.end(), query) - v.begin();
    // usar qtd_menores
}
```

20. Desafios

1. Ordene uma lista de alunos por nota (decrecente), empate por nome (alfabética).
2. Para cada consulta X, responda quantos elementos do vetor são exatamente iguais a X (usando upper_bound - lower_bound).
3. Implemente `lower_bound` manualmente e compare com a versão STL.

4. Remova duplicatas de um vetor mantendo ordenado.
5. Ordene pares por soma dos elementos.

21. Mini-projeto

Sistema de ranking: dada uma lista de (nome, pontuação), ordene por pontuação decrescente e, para uma pontuação nova, responda em qual posição do ranking ela entraria (usando busca binária na lista já ordenada).

22. Revisão

- ☒ Qual a diferença entre `lower_bound` e `upper_bound`?
- ☒ Por que `binary_search` exige vetor ordenado?
- ☒ Como ordenar por dois critérios diferentes?

23. Flashcards

Q: Como descobrir quantas vezes um valor X aparece num vetor ordenado? A:

`upper_bound(...) - lower_bound(...)`

Q: Complexidade de `sort`? A: $O(N \log N)$

24. Resumo final

`sort` organiza em $O(N \log N)$. Depois de ordenado,

`lower_bound`/`upper_bound`/`binary_search` respondem em $O(\log N)$. `pair`/`tuple`/`struct`

+ comparador customizado permitem ordenar por critérios complexos. Erro clássico: usar busca binária em vetor não ordenado.

25. Checklist de maestria

Conhecimento

- ☐ Sei explicar a diferença entre `lower_bound` e `upper_bound`
- ☐ Sei por que preciso ordenar antes de usar busca binária da STL

Implementação

- ☐ Consigo escrever um comparador customizado (lambda ou função)
- ☐ Consigo usar `pair`/`tuple` com structured bindings

Reconhecimento

- ☐ Reconheço "ordene e depois processe" como padrão de `sort` + `binary search`

Competição

- ☐ Resolvi pelo menos 3 problemas usando `sort` + `lower_bound`/`upper_bound`